PROJECT 1

SEARCH IN SNAKE

IF5150 Advance Artificial Intelligence



By:

23526015 Zayed Ramadan Rahmat

23526032 Ibnu Fikri

23526701 Rabab Fawaz Saleh Alali

School of Electrical Engineering and Informatics

Bandung Institute of Technology

2026

Table of Contents

1. Introduction....3
1.1. Background and objectives....3
1.2. Scope....3
2. Single Player Mode....3
2.1. Problem formulation....3
2.2. Algorithm implementation....5
2.2.1. Uniform Cost Search (UCS)....5
2.2.2. Breadth-First Search (BFS)....6
2.2.3. Greedy Best-First Search....7
2.2.4. A*....8
2.2.5. Heuristic....9
2.2.6. Bonus: Iterative Deepening Search (IDS)....10
2.3. Experiments and results....12
2.3.1. Test setup....12
2.3.2. Results table (Each member supplies their own algorithm's numbers)....13
2.4. Analysis....13
2.4.1. BFS vs UCS (Rabab + Ibnu Fikri)....13
2.4.2. Greedy vs A*....14
2.4.3. How the heuristic affects A*....15
2.4.4. Solution quality vs computational cost....15
2.4.5. Cases where an algorithm fails or performs poorly....16
3. Multiplayer Mode....17
3.1. Adversarial problem formulation (Rabab + Ibnu Fikri)....17
3.2. Algorithm implementation....20
3.2.1. Minimax....20
3.2.2. Alpha-Beta Pruning....23
3.2.3. Bonus: Monte Carlo Tree Search (MCTS)....27
3.3. Experiments and results....27
3.3.1. Test setup....27
3.3.2. Results table....27
3.4. Analysis of search depth....28
4. Conclusion....29
5. Member Contributions....29

1. Introduction

1.1. Background and objectives

This report covers Project 1 of IF5150 Advanced Artificial Intelligence. The project uses a Snake game framework in which a snake moves one cell per turn on a grid and must eat apples without hitting a wall or its own body. All apples are placed at the start of the game. In Single Player Mode, one snake collects the apples, which is treated as a search problem. In Multiplayer Mode, two snakes compete, which is treated as an adversarial game.

The objectives are to implement search algorithms for both modes in the framework, to compare them by experiment, and to explain how the characteristics of each algorithm affect the behaviour and performance of the snake.

1.2. Scope

This project covers both game modes of the Snake framework. In Single Player Mode, four of the five listed algorithms are implemented and compared: Breadth-First Search (BFS), Uniform Cost Search (UCS), Greedy Best-First Search and A*. Iterative Deepening Search (IDS) is added as a bonus algorithm. Depth-First Search (DFS) is not implemented.

The specification requires four of the five algorithms, and the teaching assistants confirmed that implementing and comparing four algorithms is sufficient.

In Multiplayer Mode, Minimax and Alpha-Beta Pruning are implemented and evaluated at three or more search depths. Monte Carlo Tree Search (MCTS) is added as a bonus algorithm. The search algorithms are implemented in snake_game/strategies.py, the file designated for search strategies. The game engine and user interface are not modified.

2. Single Player Mode

2.1. Problem formulation

Single Player Mode is formulated as a search problem, as defined in Table 2.1. BFS, UCS, Greedy Best-First Search and A* use the same formulation so that their results can be compared directly.

Table 2.1

Single Player Search Problem

Component	Definition
State	Head position, body cells from head to tail, and the remaining apples.
Initial state	The game state given by the framework when a search starts.
Actions	UP, DOWN, LEFT, RIGHT, excluding moves that reverse into the neck, leave the board or enter the body. The tail cell may be entered when the snake is not growing.
Transition model	The head moves one cell. If the new cell holds an apple, the snake grows by one and the apple is removed; otherwise the tail cell is freed.
Goal test	The last move eats an apple.
Step cost	1 per move, so path cost equals path length.

The body is part of the state because the snake cannot enter its own body and the body shifts with every move. To check whether a state was already visited, however, only the head position is compared. Comparing the whole body made the searches exceed the framework's 5-second limit per move.

Each search ends at the first apple reached. A new search runs every turn, until all apples are collected or the game ends. A single search over all apples was not used, since a path that collects every apple is too long to find within the framework's 5-second limit per move. Because every move costs 1, UCS is expected to find paths of the same length as BFS; this is examined in Section 2.4.

The strategy uses the search result as follows:

- Each turn, it returns the first move of the path found.
- If no path to an apple is found, it returns the first legal move, or its current direction when no legal move remains.

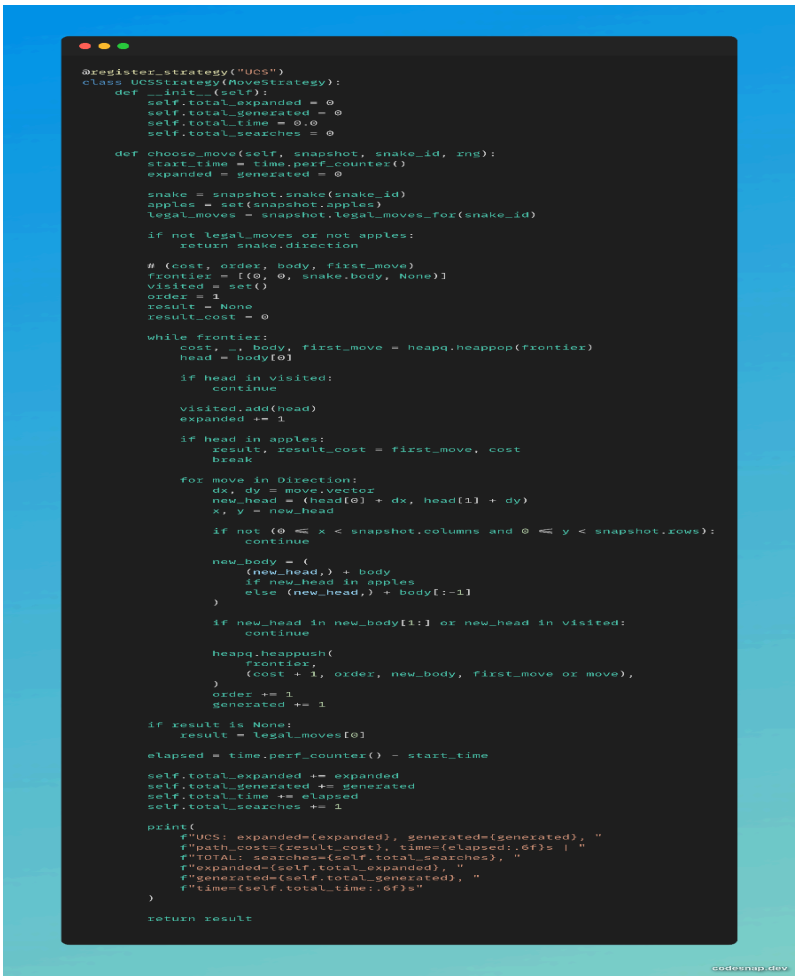
2.2. Algorithm implementation

2.2.1. Uniform Cost Search (UCS)

Uniform Cost Search (UCS) is implemented in the UCSStrategy class in snake_game/strategies.py. At each turn, UCS searches from the current snake position toward an apple using a priority queue. Since every movement has a step cost of 1, states with the lowest accumulated path cost $g(n)$ are expanded first. Invalid moves that hit the wall or the snake's body are skipped. The search stops when an apple is reached, and the first move of the resulting path is returned to the game.

Figure 2.1

Uniform Cost Search (UCS) code



2.2.2. Breadth-First Search (BFS)

Breadth-First Search (BFS) is implemented in the BFSStrategy class in snake_game/strategies.py. At each turn, BFS searches for a path from the snake's current state to an apple using a first-in, first-out (FIFO) queue. States are expanded in the order they are discovered, ensuring that states at a smaller depth are explored before states at a greater depth.

For each expanded state, BFS generates possible successor states by considering the four movement directions. Moves that leave the board or cause the snake to collide with its own body are discarded. When the snake reaches an apple, the search stops and returns the first move of the discovered path. If no path to an apple is found, the strategy returns the first legal move, or the current direction if no legal moves remain.

```python
@register_strategy("BFS")
class BFSStrategy(MoveStrategy):
    def __init__(self):
        self.total_expanded = 0
        self.total_generated = 0
        self.total_time = 0.0
        self.total_searches = 0

    def choose_move(self, snapshot, snake_id, rng):
        start_time = time.perf_counter()
        expanded = generated = 0

        snake = snapshot.snake(snake_id)
        apples = set(snapshot.apples)
        legal_moves = snapshot.legal_moves_for(snake_id)

        if not legal_moves or not apples:
            return snake.direction

        frontier = deque([(snake.body, None, 0)])

        visited = set()

        result = None
        result_cost = 0

        while frontier:
            body, first_move, cost = frontier.popleft()
            head = body[0]

            if head in visited:
                continue

            visited.add(head)

            expanded += 1

            if head in apples:
                result = first_move
                result_cost = cost
                break

            for move in Direction:
                dx, dy = move.vector
                new_head = (
                    head[0] + dx,
                    head[1] + dy
                )
                x, y = new_head

                if not (
                    0 <= x < snapshot.columns
                    and
                    0 <= y < snapshot.rows
                ):
                    continue


                new_body = (
                    (new_head,) + body
                    if new_head in apples
                    else (new_head,) + body[:-1]
                )

                if new_head in new_body[1:]:
                    continue

                # State sudah pernah dikunjungi
                if new_head in visited:
                    continue

                frontier.append(
                    (
                        new_body,
                        first_move or move,
                        cost + 1
                    )
                )
                generated += 1

        if result is None:
            result = legal_moves[0]

        elapsed = time.perf_counter() - start_time
        self.total_expanded += expanded
        self.total_generated += generated
        self.total_time += elapsed
        self.total_searches += 1
        print(
            f"BFS: expanded={expanded}, generated={generated}, "
            f"path_cost={result_cost}, time={elapsed:.6f}s | "
            f"TOTAL: searches={self.total_searches}, "
            f"expanded={self.total_expanded}, "
            f"generated={self.total_generated}, "
            f"time={self.total_time:.6f}s"
        )
        return result
```

2.2.3. Greedy Best-First Search

Greedy Best-First Search is implemented in the GreedyBestFirstStrategy class in snake_game/strategies.py (Figure 2.2). Each turn, it searches from the current game state. The frontier is sorted by h(n) (Section 2.2.5), so the state closest to an apple is expanded first. Each state has up to four successors, one per move; moves into a wall or the body are skipped. The search stops as soon as a successor reaches an apple, and the first move of that path is returned.

Figure 2.2

Greedy Best-First Search Code

```python
@register_strategy("Greedy Best-First")
class GreedyBestFirstStrategy(MoveStrategy):
    def choose_move(self, snapshot, snake_id, rng):
        import time
        start_time = time.perf_counter()
        expanded = 0
        generated = 0

        snake = snapshot.snake(snake_id)
        apples = snapshot.apples

        def h(head):
            return min(abs(head[0] - ax) + abs(head[1] - ay) for ax, ay in apples)

        # each frontier item: (body, first_move)
        frontier = [(snake.body, None)]
        visited = {snake.body[0]}
        result = None

        while frontier and apples and result is None:
            frontier.sort(key=lambda item: h(item[0][0]))
            body, first_move = frontier.pop(0)
            expanded += 1

            for move in Direction:
                dx, dy = move.vector
                new_head = (body[0][0] + dx, body[0][1] + dy)
                if not (0 <= new_head[0] < snapshot.columns and 0 <= new_head[1] < snapshot.rows):
                    continue
                if new_head in body[:-1]:
                    continue
                new_body = (new_head,) + body[:-1]
                if new_head in visited:
                    continue
                visited.add(new_head)
                generated += 1

                if first_move is None:
                    first_move_here = move
                else:
                    first_move_here = first_move
                if new_head in apples:
                    result = first_move_here
                    break
                frontier.append((new_body, first_move_here))

        if result is None:
            legal_moves = snapshot.legal_moves_for(snake_id)
            result = legal_moves[0] if legal_moves else snake.direction

        elapsed = time.perf_counter() - start_time
        print(f"Greedy Best-First: expanded={expanded}, generated={generated}, time={elapsed:.4f}s")
        return result
```

2.2.4. A*

A* is implemented in the AStarStrategy class in the same file (Figure 2.3). It works like Greedy Best-First Search, with two differences. The frontier is sorted by f(n) = g(n) + h(n), where g(n) is the number of moves from the current state to n. The goal test is done when a state is taken from the frontier, not when it is created. With an admissible h(n), this gives a shortest path, but it is not guaranteed here, since visited states are compared by head position only (Section 2.1).

Figure 2.3

A Code*

```python
@register_strategy("A*")
class AStarStrategy(MoveStrategy):
    def choose_move(self, snapshot, snake_id, rng):
        import time
        start_time = time.perf_counter()
        expanded = 0
        generated = 0

        snake = snapshot.snake(snake_id)
        apples = snapshot.apples

        def h(head):
            return min(abs(head[0] - ax) + abs(head[1] - ay) for ax, ay in apples)

        # each frontier item: (body, first_move, g)
        frontier = [(snake.body, None, 0)]
        visited = set()
        result = None

        while frontier and apples:
            frontier.sort(key=lambda item: item[2] + h(item[0][0]))
            body, first_move, g = frontier.pop(0)
            if body[0] in visited:
                continue
            visited.add(body[0])

            if body[0] in apples:
                result = first_move
                break
            expanded += 1

            for move in Direction:
                dx, dy = move.vector
                new_head = (body[0][0] + dx, body[0][1] + dy)
                if not (0 <= new_head[0] < snapshot.columns and 0 <= new_head[1] < snapshot.rows):
                    continue
                if new_head in body[:-1]:
                    continue
                new_body = (new_head,) + body[:-1]
                if new_head in visited:
                    continue
                generated += 1

                if first_move is None:
                    first_move_here = move
                else:
                    first_move_here = first_move
                frontier.append((new_body, first_move_here, g + 1))

        if result is None:
            legal_moves = snapshot.legal_moves_for(snake_id)
            result = legal_moves[0] if legal_moves else snake.direction

        elapsed = time.perf_counter() - start_time
        print(f"A*: expanded={expanded}, generated={generated}, time={elapsed:.4f}s")
        return result
```

2.2.5. Heuristic

Greedy Best-First Search and A* use the same heuristic: the Manhattan distance from the head to the nearest remaining apple. For head position (x, y), h(n) = mina (|x − xa| + |y − ya|), taken over every remaining apple at (xa, ya). It uses only the head and apple positions, so it is cheap to compute at every node.

The heuristic is admissible. Each move shifts the head one cell and costs 1 (Table 2.1), so the head needs at least h(n) moves to reach any apple. The body and the board edges can only make the real path longer. It is also consistent, since one move changes h(n) by at most 1. As a

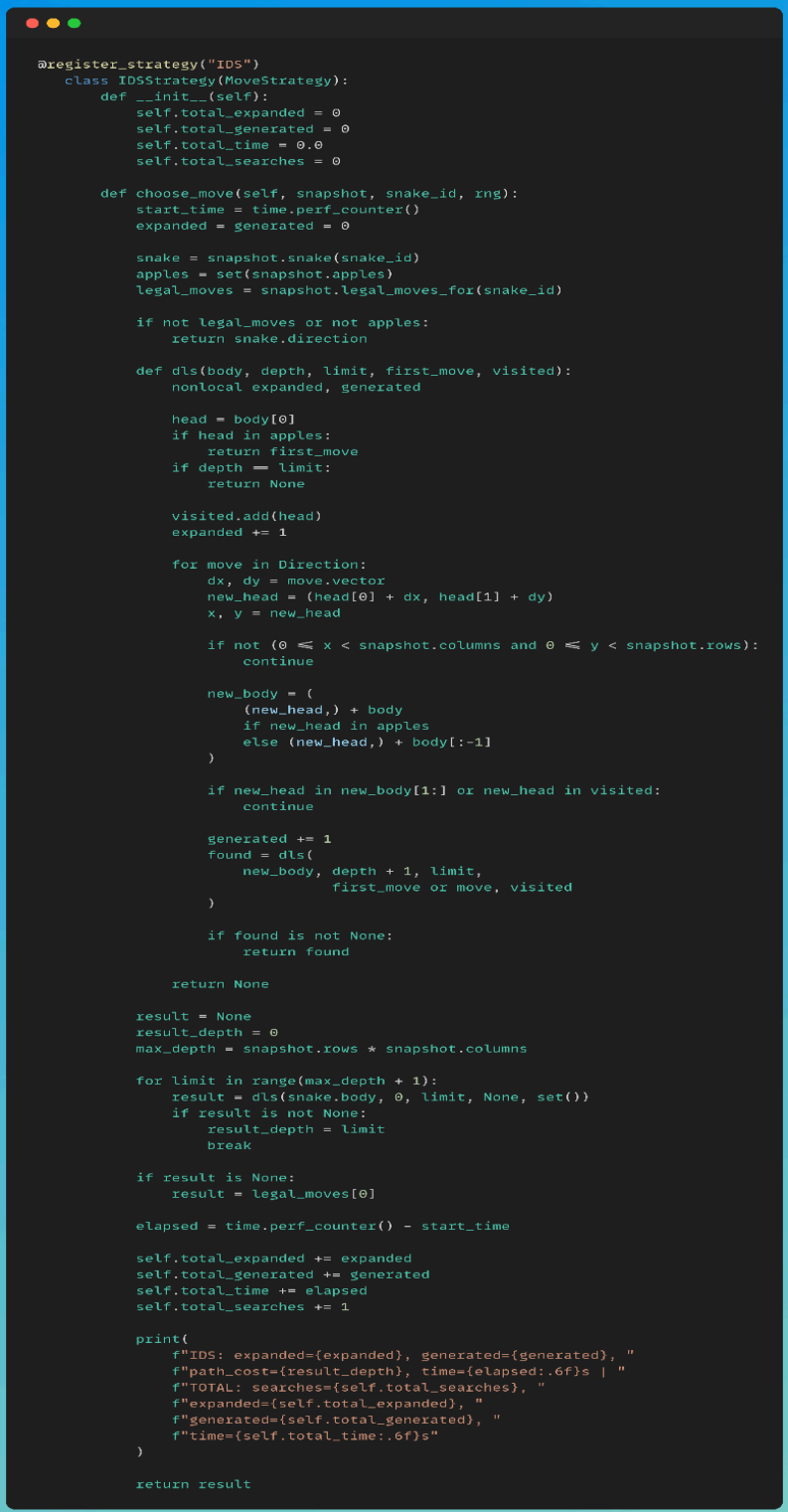
result, A* finds a shortest path to the closest reachable apple, the same length that BFS and UCS find, while Greedy Best-First Search may not.

2.2.6. Bonus: Iterative Deepening Search (IDS)

Iterative Deepening Search (IDS) is implemented in the IDSStrategy class in snake_game/strategies.py. Each turn, IDS repeatedly performs a depth-limited search, starting with depth 0 and gradually increasing the depth limit until a path to an apple is found. At each depth, the algorithm explores possible movements while skipping moves that would hit the wall or the snake's body. When an apple is reached, the search stops and the first move of the discovered path is returned. This allows IDS to combine the low memory usage of depth-first search with the ability to find solutions at increasing depths.

Figure 2.4

Code Iterative Deepening Search (IDS)



```
@register_strategy("IDS")
class IDSStrategy(MoveStrategy):
    def __init__(self):
        self.total_expanded = 0
        self.total_generated = 0
        self.total_time = 0.0
        self.total_searches = 0

    def choose_move(self, snapshot, snake_id, rng):
        start_time = time.perf_counter()
        expanded = generated = 0

        snake = snapshot.snake(snake_id)
        apples = set(snapshot.apples)
        legal_moves = snapshot.legal_moves_for(snake_id)

        if not legal_moves or not apples:
            return snake.direction

        def dls(body, depth, limit, first_move, visited):
            nonlocal expanded, generated

            head = body[0]
            if head in apples:
                return first_move
            if depth == limit:
                return None

            visited.add(head)
            expanded += 1

            for move in Direction:
                dx, dy = move.vector
                new_head = (head[0] + dx, head[1] + dy)
                x, y = new_head

                if not (0 <= x < snapshot.columns and 0 <= y < snapshot.rows):
                    continue

                new_body = (
                    (new_head,) + body
                    if new_head in apples
                    else (new_head,) + body[:-1]
                )

                if new_head in new_body[1:] or new_head in visited:
                    continue

                generated += 1
                found = dls(
                    new_body, depth + 1, limit,
                              first_move or move, visited
                )

                if found is not None:
                    return found

            return None

        result = None
        result_depth = 0
        max_depth = snapshot.rows * snapshot.columns

        for limit in range(max_depth + 1):
            result = dls(snake.body, 0, limit, None, set())
            if result is not None:
                result_depth = limit
                break

        if result is None:
            result = legal_moves[0]

        elapsed = time.perf_counter() - start_time

        self.total_expanded += expanded
        self.total_generated += generated
        self.total_time += elapsed
        self.total_searches += 1

        print(
            f"IDS: expanded={expanded}, generated={generated}, "
            f"path_cost={result_depth}, time={elapsed:.6f}s | "
            f"TOTAL: searches={self.total_searches}, "
            f"expanded={self.total_expanded}, "
            f"generated={self.total_generated}, "
            f"time={self.total_time:.6f}s"
        )

        return result
```

codesnap.dev

2.3. Experiments and results

2.3.1. Test setup

All Single Player experiments use the default settings of the framework: a 20 × 20 grid, 5 apples and a limit of 200 moves. The snake starts at the centre of the board with length 3, facing right. Apple positions change in every game because the framework sets no fixed seed. Each algorithm therefore plays 20 games, and the results are averaged over these games. The 5-second limit per move applies to every algorithm, and a move that exceeds it ends the game as a failure.

The framework shows the score, the number of moves and the reason the game ended. These give food collected, success rate, path length and path cost. States expanded, states generated and search time are not shown, since the elapsed time on screen is the total game time, so each algorithm counts them during its search and prints them. The numbers are then copied into the results table by hand. Table 2.2 defines each metric.

Table 2.2

Single Player Metrics

Metric	Definition
Food collected	Apples eaten in a game, from 0 to 5. The framework lowers the score by 1 when the game ends in a collision or a strategy failure, so 1 is added back in those games.
Success rate	Share of the 20 games in which all 5 apples are eaten.
States expanded	States taken from the frontier and expanded, summed over all searches in a game.
States generated	Successor states created, summed over all searches in a game.
Path length	Moves made in a game.
Path cost	Sum of step costs in a game. It equals path length because each move costs 1.
Execution time	Time spent searching during a game.

2.3.2. Results table (Each member supplies their own algorithm's numbers)

Table 2.3 gives the results over the 20 games of each algorithm. Success rate is a share of the 20 games; the other metrics are averages per game.

Table 2.3

Single Player Results

Algorithm	Food collected	Success rate	States expanded	States generated	Path length	Path cost	Execution time (s)
BFS	5.00	100%	3583.36	7453.64	42.56	42.56	0.0363
UCS	5.00	100%	4125.4	8465.35	44.45	44.45	0.0614
Greedy Best-First Search	5.00	100%	313.9	860.9	41.70	41.70	0.0053
A*	5.00	100%	801.0	1870.2	44.40	44.40	0.0386
IDS (bonus)	5.00	100%	129576.75	213256.95	70.15	70.15	1.2377

2.4. Analysis

2.4.1. BFS vs UCS (Rabab + Ibnu Fikri)

Both BFS and UCS collected all five apples in all 20 games, achieving a 100% success rate. However, their computational costs differed. BFS expanded an average of 3,583.36 states and generated 7,453.64 states per game, while UCS expanded 4,125.40 states and generated 8,465.35 states. BFS also had a lower average execution time of 0.0363 seconds per game, compared with 0.0614 seconds for UCS.

The difference can be explained by the way the two algorithms organize their frontiers. BFS uses a FIFO queue and expands states in increasing order of depth. UCS uses a priority queue and expands the state with the lowest accumulated path cost, g(n). In this project, every movement has a step cost of 1, so path cost is equal to path length. Therefore, BFS and UCS are expected to find paths with the same minimum cost under the same search-state representation.

Although their theoretical search objectives are equivalent in this setting, their measured results are not necessarily identical. BFS and UCS may expand states in different orders when several states have the same path cost. In addition, the implementation marks states as visited based only on the snake's head position, even though the complete body configuration affects which moves are legal. This simplification can influence the paths found by both algorithms.

Overall, BFS was more computationally efficient than UCS in these experiments, as shown by its lower numbers of expanded and generated states and its shorter execution time. Both algorithms achieved the same success rate, indicating that UCS's priority-queue ordering did not provide an observable advantage under the tested conditions. These findings apply to the tested games and do not imply that BFS is always faster than UCS in every search problem.

2.4.2. Greedy vs A*

Both algorithms ate all 5 apples in all 20 games (Section 2.3.2). Greedy Best-First Search was cheaper. Per game, it expanded 313.9 states and generated 860.9 on average, against 801.0 and 1870.2 for A*. Its search time was 0.0053 s per game, against 0.0386 s for A*. Greedy Best-First Search follows h(n) only and stops at the first successor that reaches an apple. A* also counts g(n), so it expands states on several routes of equal length (Section 2.4.3).

A* is expected to make fewer moves, yet its average path was longer, 44.40 moves against 41.70. Apple positions differed in every game, and A* finds the shortest path to the nearest apple at each turn, not the shortest route through all apples. The difference in path length therefore says little about solution quality.

2.4.3. How the heuristic affects A*

The heuristic sets the order in which A* expands states. With h(n) = 0, A* sorts the frontier by g(n) alone and behaves like UCS, which expands states in rings of growing distance from the head, in every direction. With the Manhattan distance, states on the side of the nearest apple get a lower f(n) and are expanded first, so the search moves toward that apple. Since h(n) is admissible and consistent, it does not make A* miss a shorter path, apart from the effect of the head-only visited check.

The heuristic does not make A* as cheap as Greedy Best-First Search. A* still expands every state whose f(n) is below the length of the shortest path, and some whose f(n) equals it. On an open board, many cells between the head and the apple have an f(n) equal to that length, so A* expands states on several routes of equal length. Greedy Best-First Search ignores g(n) and follows one route.

2.4.4. Solution quality vs computational cost

The experimental results show that a higher computational cost does not always lead to better solution quality. UCS, Greedy Best-First Search, A*, and IDS all collected 5 apples in all 20 games, resulting in a 100% success rate. However, the amount of computation required by each algorithm was very different.

Greedy Best-First Search had the lowest computational cost, with an average of 313.9 expanded states and an execution time of 0.0053 seconds per game. A* required more computation, while UCS expanded an average of 4,125.4 states with an average execution time of 0.0614 seconds. IDS had the highest computational cost, expanding 129,576.75 states on average and taking 1.2377 seconds per game. Its average path length was also the highest at 70.15 moves.

These results suggest that, for this experiment, using more computational resources did not necessarily improve the final result. All algorithms achieved the same success rate, while Greedy reached this result with much lower computational cost. However, the apple positions were different between games, so the path lengths and execution times should be interpreted as general

performance trends rather than an exact direct comparison between identical game states.(need ibnu's results)

2.4.5. Cases where an algorithm fails or performs poorly

BFS may perform poorly when the search space becomes large because it explores states level by level and stores unexplored states in a queue. This can lead to high memory consumption and increased execution time, especially when the snake must navigate a large grid or search for distant apples. Another limitation occurs when the implementation identifies visited states only by the snake's head position. Two states with the same head position but different body configurations may have different legal moves. Treating them as the same state can cause BFS to overlook a valid path or produce an invalid path.

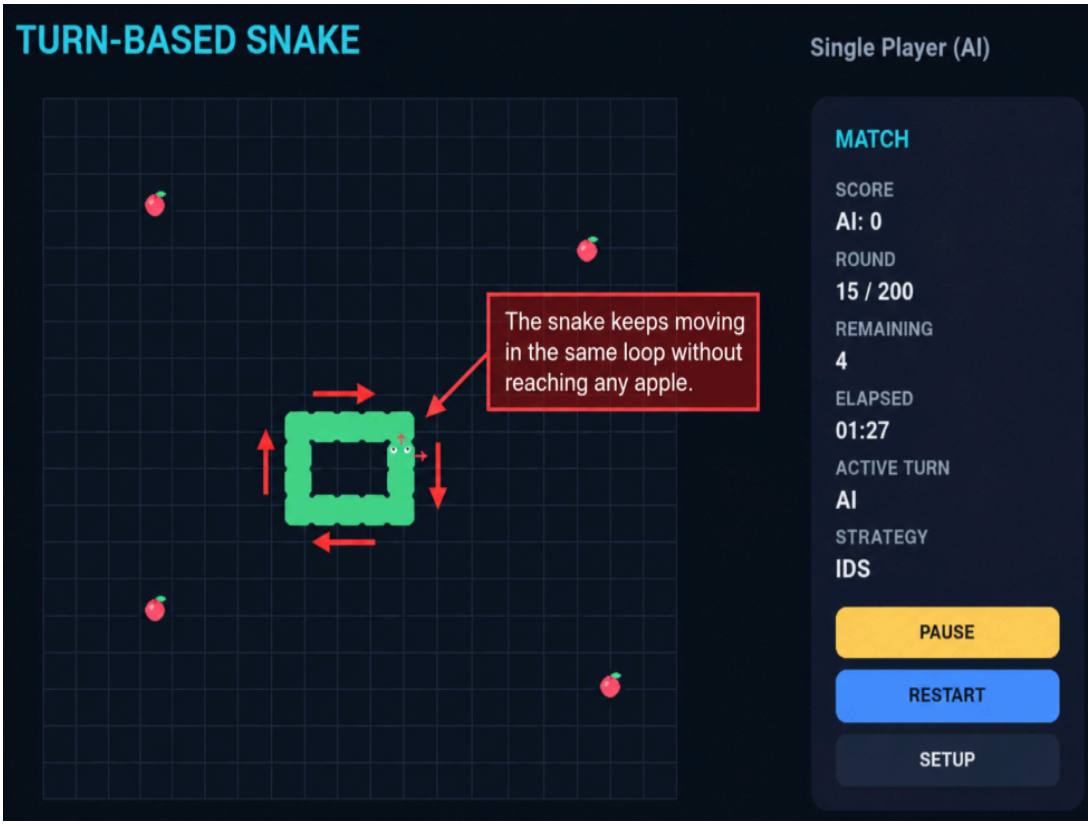
Alpha-Beta Pruning may become computationally expensive when the search depth or branching factor increases. Although pruning reduces the number of states explored, the algorithm may still need to examine many states, particularly when move ordering is ineffective and few branches can be pruned. Its performance also depends on the quality of the evaluation function. If the function prioritizes score differences and distance to apples but does not adequately consider survival, trapped positions, or future mobility, the algorithm may select a move that appears beneficial in the short term but leads to a disadvantageous position later. Finally, Alpha-Beta Pruning is limited by its search horizon. The algorithm may fail to recognize a danger or opportunity that occurs beyond the selected search depth. Increasing the depth can help anticipate more future moves, but it also increases computational cost.

In one IDS experiment, the snake became stuck in a repetitive movement pattern and repeatedly moved through similar positions before eventually completing the game. This run required 200 moves, expanded

1,422,449 states, generated 2,220,943 states, and took approximately 15.65 seconds. This was considerably more expensive than the other IDS runs and shows how IDS can perform poorly in certain game configurations.

The repeated depth-limited searches, together with the simplified visited-state representation based only on the snake's head position, can result in inefficient exploration and repetitive movement.

Figure 2.5



3. Multiplayer Mode

3.1. Adversarial problem formulation (Rabab + Ibnu Fikri)

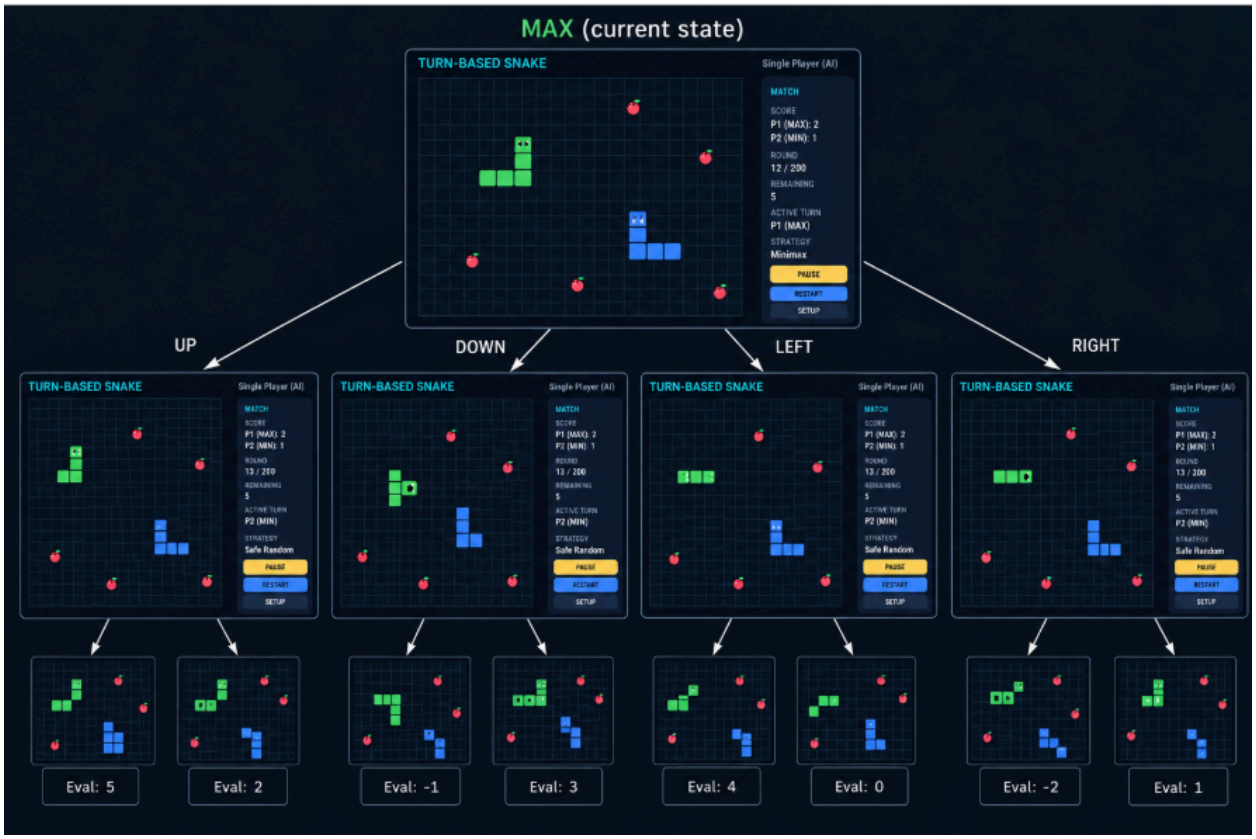
In multiplayer mode, the Snake game involves two snakes competing to collect apples on the same board. This is treated as an adversarial search problem, where one snake acts as the maximizing player (MAX) and the other as the minimizing player (MIN). Both snakes take turns

moving, and each player needs to consider the possible actions of the opponent before choosing its next move.

The game state includes the body positions, movement directions, and scores of both snakes, along with the remaining apples. Each snake can move up, down, left, or right, but moves that hit a wall, collide with a snake's body, or reverse the current direction are not allowed. When a snake moves, its body position is updated. If it reaches an apple, its score increases and its body grows.

Figure 3.1 illustrates how the multiplayer game is represented as a search tree. MAX explores possible moves and tries to select the highest evaluation value, while MIN is assumed to choose the lowest value. Since exploring every possible move would require too much computation, the search is limited to a certain depth. The remaining states are evaluated based on the difference in scores, distance to the nearest apple, and number of available legal moves. This allows the algorithm to estimate which move is more beneficial while also considering the opponent's possible response.

Figure 3.1



The search tree alternates between MAX and MIN decisions. At each MAX level, the algorithm selects the action with the highest value. At each MIN level, it selects the action with the lowest value. Since the complete game tree can be too large to explore, the search is limited to a predefined depth.

At the depth limit, the evaluation function estimates the desirability of the current state. It considers the score difference between the snakes and their Manhattan distances to the nearest remaining apple. A positive score difference favours MAX, while a shorter distance to an apple generally improves a snake's opportunity to collect it. This formulation allows the search algorithms to consider both the controlled snake's objectives and the opponent's possible responses.

Minimax explores the game tree using this evaluation process. Alpha-Beta Pruning applies the same minimax decision principle but avoids exploring branches that cannot affect the final decision. Thus, both algorithms can select the same move at an equivalent search depth, while Alpha-Beta Pruning may require less computation.

3.2. Algorithm implementation

3.2.1. Minimax

Minimax is implemented in the MinimaxStrategy class (Figure 3.2). It is used for the multiplayer Snake game, where the controlled snake acts as the maximizing player (MAX) and the opponent acts as the minimizing player (MIN).

For each turn, the algorithm explores the possible moves of both snakes up to a fixed search depth. When the depth limit is reached, the state is evaluated using the score difference, distance to the nearest apple, and the number of legal moves available to each snake. MAX chooses the move with the highest value, while MIN chooses the move with the lowest value. The move with the best final Minimax value is then selected.

Figure 3.2

Code Minimax



```
@register_strategy("Minimax")
        class MinimaxStrategy(MoveStrategy):
            MAX_DEPTH = 4

            def choose_move(self, snapshot, snake_id, rng):
                start_time = time.perf_counter()
                generated = 0
                evaluated = 0

                snake = snapshot.snake(snake_id)
                legal_moves = snapshot.legal_moves_for(snake_id)

                if not legal_moves:
                    return snake.direction

                opponent = next(
                    (s for s in snapshot.snakes if s.snake_id != snake_id),
                    None
                )

                if opponent is None:
                    return legal_moves[0]

                opponent_id = opponent.snake_id
                apples = set(snapshot.apples)

                state = {
                    s.snake_id: {
                        "body": tuple(s.body),
                        "direction": s.direction,
                        "score": s.score
                    }
                    for s in snapshot.snakes
                }

                def get_moves(state, apples, player):
                    body = state[player]["body"]
                    direction = state[player]["direction"]
                    head = body[0]
                    possible_moves = []

                    for move in Direction:
                        if len(body) > 1 and move is direction.opposite:
                            continue

                        dx, dy = move.vector
                        new_head = (head[0] + dx, head[1] + dy)
                        x, y = new_head

                        if not (0 <= x < snapshot.columns and
                                0 <= y < snapshot.rows):
                            continue

                        own_body = body if new_head in apples else body[:-1]

                        if new_head in own_body:
                            continue

                        other_id = (
                            opponent_id if player == snake_id else snake_id
                        )
                        other_body = state[other_id]["body"]

                        if new_head in other_body:
                            continue

                        possible_moves.append(move)

                    return possible_moves

                def evaluate(state, apples):
                    me = state[snake_id]
                    opponent = state[opponent_id]

                    score = (me["score"] - opponent["score"]) * 1000

                    if not apples:
                        return score

                    my_head = me["body"][0]
                    opponent_head = opponent["body"][0]

                    my_distance = min(
                        abs(my_head[0] - x) + abs(my_head[1] - y)
                        for x, y in apples
                    )

                    opponent_distance = min(
                        abs(opponent_head[0] - x) + abs(opponent_head[1] - y)
                        for x, y in apples
                    )

                    score += (opponent_distance - my_distance) * 20

                    my_moves = len(
                        get_moves(state, apples, snake_id)
                    )
                    opponent_moves = len(
                        get_moves(state, apples, opponent_id)
                    )

                    score += (my_moves - opponent_moves) * 5

                    return score
```

codesnap.dev

```
def minimax(state, apples, player, depth):
                    nonlocal generated, evaluated

                    if depth == 0 or not apples:
                        evaluated += 1
                        return evaluate(state, apples)

                    moves = get_moves(state, apples, player)

                    if not moves:
                        evaluated += 1
                        return -10000 if player == snake_id else 10000

                    next_player = (
                        opponent_id if player == snake_id else snake_id
                    )

                    values = []

                    for move in moves:
                        new_state, new_apples = make_move(
                            state, apples, player, move
                        )

                        generated += 1

                        value = minimax(
                            new_state,
                            new_apples,
                            next_player,
                            depth - 1
                        )

                        values.append(value)

                    if player == snake_id:
                        return max(values)

                    return min(values)

                best_move = legal_moves[0]
                best_value = float("-inf")

                for move in legal_moves:
                    new_state, new_apples = make_move(
                        state, apples, snake_id, move
                    )

                    generated += 1

                    value = minimax(
                        new_state,
                        new_apples,
                        opponent_id,
                        self.MAX_DEPTH - 1
                    )

                    if value > best_value:
                        best_value = value
                        best_move = move

                elapsed = time.perf_counter() - start_time

                print(
                    f"Minimax: depth={self.MAX_DEPTH}, "
                    f"generated={generated}, "
                    f"evaluated={evaluated}, "
                    f"value={best_value:.2f}, "
                    f"move={best_move.name}, "
                    f"time={elapsed:.6f}s"
                )

                return best_move
```

codesnap.dev

3.2.2. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of the Minimax algorithm used in adversarial search. It maintains two bounds: alpha, the best value found so far for the maximizing player, and beta, the best value found so far for the minimizing player. When alpha becomes greater than or equal to beta, the remaining branches can be pruned because they cannot affect the final decision. Alpha-Beta Pruning can therefore reduce the number of states evaluated while preserving Minimax's decision for the same search tree and evaluation function. Its effectiveness depends on the search depth, branching factor, and move ordering.

```
@register_strategy("Alpha-Beta")
class AlphaBetaStrategy(MoveStrategy):
    MAX_DEPTH = 1
    def __init__(self):
        self.total_states_generated = 0
        self.total_states_evaluated = 0
        self.total_states_pruned = 0
        self.total_elapsed = 0
    def choose_move(self, snapshot: GameSnapshot, snake_id: str, rng: Random) -> Direction:
        start_time = time.perf_counter()
        states_generated = 0
        states_evaluated = 0
        states_pruned = 0
        legal_moves = snapshot.legal_moves_for(snake_id)
        snake = snapshot.snake(snake_id)

        if not legal_moves:
            return snake.direction

        opponent = next((other for other in snapshot.snakes if other.snake_id != snake_id),None,)

        if opponent is None:
            return legal_moves[0]

        opponent_id = opponent.snake_id

        snakes = {current.snake_id: {"body": tuple(current.body), "direction": current.direction,
                "score": current.score,} for current in snapshot.snakes}

        apples = set(snapshot.apples)

        def evaluate(state, remaining_apples):
```

```
            me = state[snake_id]
            other = state[opponent_id]

            score_difference = me["score"] - other["score"]

            if not remaining_apples:
                return score_difference * 1000

            my_head = me["body"][0]
            other_head = other["body"][0]

            my_distance = min(abs(my_head[0] - x) + abs(my_head[1] - y) for x, y in remaining_apples)

            other_distance = min(abs(other_head[0] - x) + abs(other_head[1] - y) for x, y in
remaining_apples)

            return (score_difference * 100 + other_distance - my_distance)

        def get_legal_moves(state, remaining_apples, current_id):
            current = state[current_id]
            body = current["body"]
            head = body[0]

            moves = []

            for move in Direction:
                if (len(body) > 1 and move is current["direction"].opposite):
                    continue

                dx, dy = move.vector
                new_head = (head[0] + dx, head[1] + dy)

                x, y = new_head

                # Cek batas board
                if not (0 <= x < snapshot.columns and 0 <= y < snapshot.rows):
                    continue

                growing = new_head in remaining_apples

                own_body = body if growing else body[:-1]

                # Cek tabrakan dengan tubuh sendiri
                if new_head in own_body:
                    continue

                # Cek tabrakan dengan snake lain
                hits_other_snake = any(new_head in other["body"] for other_id, other in state.items() if
other_id != current_id)

                if hits_other_snake:
                    continue

                moves.append(move)
            return moves

        def apply_move(state, remaining_apples, current_id, move):
            new_state = {sid: {
                    "body": tuple(data["body"]),
```

```
                "direction": data["direction"],
                "score": data["score"],}
            for sid, data in state.items()}

        new_apples = set(remaining_apples)

        current = new_state[current_id]
        body = current["body"]

        dx, dy = move.vector

        head_x, head_y = body[0]

        new_head = (head_x + dx, head_y + dy)

        # Jika mendapatkan apple
        if new_head in new_apples:
            new_body = ((new_head,) + body)
            new_apples.remove(new_head)
            new_score = current["score"] + 1

        # Jika tidak mendapatkan apple
        else:
            new_body = ((new_head,) + body[:-1])
            new_score = current["score"]

        new_state[current_id] = {"body": new_body, "direction": move, "score": new_score,}
        return new_state, new_apples

    def alpha_beta(state, remaining_apples, current_id, depth, alpha, beta):
        nonlocal states_generated
        nonlocal states_evaluated
        nonlocal states_pruned

        if depth == 0 or not remaining_apples:
            states_evaluated += 1
            return evaluate(state, remaining_apples)

        moves = get_legal_moves(state, remaining_apples, current_id)

        if not moves:
            states_evaluated += 1
            if current_id == snake_id:
                return -10000
            return 10000

        next_id = (opponent_id if current_id == snake_id else snake_id)

        if current_id == snake_id:
            best_value = float("-inf")
            for move in moves:
                next_state, next_apples = apply_move(state, remaining_apples, current_id, move)

                states_generated += 1

                value = alpha_beta(next_state, next_apples, next_id, depth - 1, alpha, beta)

                best_value = max(best_value, value)
```

```
                alpha = max(alpha, best_value)

                if beta <= alpha:
                    states_pruned += 1
                    break

            return best_value


        else:

            best_value = float("inf")

            for move in moves:

                next_state, next_apples = apply_move(state, remaining_apples, current_id, move)

                states_generated += 1

                value = alpha_beta(next_state, next_apples, next_id, depth - 1, alpha, beta)

                best_value = min(best_value, value)

                beta = min(beta, best_value)

                if beta <= alpha:
                    states_pruned += 1
                    break

            return best_value


    best_move = legal_moves[0]
    best_value = float("-inf")

    alpha = float("-inf")
    beta = float("inf")

    for move in legal_moves:
        next_state, next_apples = apply_move(snakes, apples, snake_id, move)

        states_generated += 1

        value = alpha_beta(next_state, next_apples, opponent_id, self.MAX_DEPTH - 1, alpha, beta)

        if value > best_value:

            best_value = value
            best_move = move

        alpha = max(
            alpha,
            best_value
        )

    elapsed = time.perf_counter() - start_time

    print(
        f"Alpha-Beta: depth={self.MAX_DEPTH}, "
```

```
            f"generated={states_generated}, "
            f"evaluated={states_evaluated}, "
            f"pruned={states_pruned}, "
            f"time={elapsed:.6f}s"
        )
        self.total_states_generated += states_generated
        self.total_states_evaluated += states_evaluated
        self.total_states_pruned += states_pruned
        self.total_elapsed += elapsed
        print(
            f"Total: Generated={self.total_states_generated},"
            f"evaluated={self.total_states_evaluated}, "
            f"pruned={self.total_states_pruned}, "
            f"time={self.total_elapsed:.6f}s"
        )

        return best_move
```

3.2.3. Bonus: Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is implemented in the MCTSStrategy class (Figure 3.3) as a bonus multiplayer algorithm. It uses the adversarial formulation in Section 3.1, and its tree alternates between the moves of the controlled snake and those of the opponent. Each turn, MCTS repeats four steps 200 times. Selection walks down the tree using UCB1 with an exploration constant of 1.41.

At the opponent's levels the average reward is negated, so the opponent is assumed to choose the move that is worst for the controlled snake. Expansion adds one untried move as a new node. Simulation plays random legal moves for both snakes for up to 20 rounds. Backpropagation adds the result to every node on the path back to the root. The reward is the change in the score difference between the two snakes during the playout, under the game's own rules: +1 for each apple and −1 for a snake left with no legal move. The move whose node was visited most often is returned.

Against Safe Random, with the settings in Section 3.3.1, MCTS won all 10 games with an average score of 4.6 against 0.4. Per move, it generated an average of 7,712.7 states and scored 200 playouts in 0.0712 seconds (Table 3.3.2).

The reward counts only apples and losses. A playout that reaches no apple within 20 rounds scores 0, so when every apple is far away, all moves receive similar averages and the choice is close to random. MCTS also cannot see beyond the 20-round playout limit.

Figure 3.3

Code MCTS

```
990+ @register_strategy("MCTS")
991+ class MCTSStrategy(MoveStrategy):
992+     ITERATIONS = 200
993+     ROLLOUT_ROUNDS = 20
994+     EXPLORATION = 1.41
995+ 
996+     def choose_move(self, snapshot, snake_id, rng):
997+         from math import log, sqrt
998+ 
999+         start_time = time.perf_counter()
1000+         states_generated = 0
1001+         states_evaluated = 0
1002+ 
1003+         legal_moves = snapshot.legal_moves_for(snake_id)
1004+         snake = snapshot.snake(snake_id)
1005+         opponent = next((other for other in snapshot.snakes if other.snake_id != snake_id), None)
1006+         if not legal_moves or opponent is None:
1007+             return legal_moves[0] if legal_moves else snake.direction
1008+         opponent_id = opponent.snake_id
1009+ 
1010+         def other_player(player_id):
1011+             return opponent_id if player_id == snake_id else snake_id
1012+ 
1013+         def score_difference(state):
1014+             return state["snakes"][snake_id]["score"] - state["snakes"][opponent_id]["score"]
1015+ 
```

```
        def get_legal_moves(state):
            player = state["snakes"][state["turn"]]
            enemy_body = state["snakes"][other_player(state["turn"])]["body"]
            head_x, head_y = player["body"][0]
            moves = []
            for move in Direction:
                if len(player["body"]) > 1 and move is player["direction"].opposite:
                    continue
                dx, dy = move.vector
                new_head = (head_x + dx, head_y + dy)
                if not (0 <= new_head[0] < snapshot.columns and 0 <= new_head[1] < snapshot.rows):
                    continue
                if new_head in state["apples"]:
                    own_body = player["body"]
                else:
                    own_body = player["body"][:-1]
                if new_head in own_body or new_head in enemy_body:
                    continue
                moves.append(move)
            return moves

        def apply_move(state, move):
            player_id = state["turn"]
            player = state["snakes"][player_id]
            dx, dy = move.vector
            new_head = (player["body"][0][0] + dx, player["body"][0][1] + dy)
            apples = state["apples"]
            score = player["score"]
            if new_head in apples:
                body = (new_head,) + player["body"]
                apples = apples - {new_head}
                score += 1
            else:
                body = (new_head,) + player["body"][:-1]
            snakes = dict(state["snakes"])
            snakes[player_id] = {"body": body, "direction": move, "score": score}
            return {"snakes": snakes, "apples": apples, "turn": other_player(player_id), "over": not apples}
```

```
        def stuck_player_loses(state):
            snakes = dict(state["snakes"])
            stuck = dict(snakes[state["turn"]])
            stuck["score"] -= 1
            snakes[state["turn"]] = stuck
            return {"snakes": snakes, "apples": state["apples"], "turn": state["turn"], "over": True}

        def random_playout(state):
            nonlocal states_generated
            for _ in range(self.ROLLOUT_ROUNDS * 2):
                if state["over"]:
                    break
                moves = get_legal_moves(state)
                if not moves:
                    state = stuck_player_loses(state)
                    break
                state = apply_move(state, rng.choice(moves))
                states_generated += 1
            return score_difference(state) - start_difference

        def new_node(state, move, parent):
            untried = [] if state["over"] else get_legal_moves(state)
            return {"state": state, "move": move, "parent": parent, "children": [],
                    "untried": untried, "visits": 0, "total": 0}
```

```
1079+        def pick_child_by_ucb1(node):
1080+            best_child = None
1081+            best_score = -inf
1082+            for child in node["children"]:
1083+                average = child["total"] / child["visits"]
1084+                if node["state"]["turn"] == opponent_id:
1085+                    average = -average  # the opponent picks the move that is worst for us
1086+                score = average + self.EXPLORATION * sqrt(log(node["visits"]) / child["visits"])
1087+                if score > best_score:
1088+                    best_score = score
1089+                    best_child = child
1090+            return best_child
1091+
1092+        start_state = {
1093+            "snakes": {
1094+                current.snake_id: {"body": tuple(current.body), "direction": current.direction, "score": current.score}
1095+                for current in snapshot.snakes
1096+            },
1097+            "apples": frozenset(snapshot.apples),
1098+            "turn": snake_id,
1099+            "over": False,
1100+        }
1101+        start_difference = score_difference(start_state)
1102+        root = new_node(start_state, None, None)
1103+
1104+        for _ in range(self.ITERATIONS):
1105+            node = root
1106+            while not node["untried"] and node["children"]:
1107+                node = pick_child_by_ucb1(node)
1108+
1109+            if node["untried"]:
1110+                move = node["untried"].pop()
1111+                child = new_node(apply_move(node["state"], move), move, node)
1112+                node["children"].append(child)
1113+                states_generated += 1
1114+                node = child
1115+
1116+            reward = random_playout(node["state"])
1117+            states_evaluated += 1
1118+
1119+            while node is not None:
1120+                node["visits"] += 1
1121+                node["total"] += reward
1122+                node = node["parent"]
1123+
1124+        best_child = max(root["children"], key=lambda child: child["visits"])
1125+
1126+        elapsed = time.perf_counter() - start_time
1127+        print(f"MCTS: iterations={self.ITERATIONS}, generated={states_generated}, "
1128+              f"evaluated={states_evaluated}, time={elapsed:.4f}s")
1129+        return best_child["move"]
1130+
```

3.3. Experiments and results

3.3.1. Test setup

The multiplayer experiments use the default settings of the framework: a 20 × 20 grid, 5 apples, and a limit of 200 rounds. Two snakes start from their predefined positions and take turns competing for apples. Minimax and Alpha-Beta Pruning are tested at three search depths: 1, 2,

and 3. Each algorithm plays 10 games at every depth against Safe Random, using the same game settings. Apple positions vary between games because no fixed random seed is used.

The framework displays the final scores and game results. These are used to calculate the average score and win rate. The number of generated states, evaluated states, pruned branches, and search execution time are measured during the search and printed in the terminal. The results are then recorded manually and averaged across the 10 games for each algorithm and depth.

3.3.2. Results table

Strategy	Depth	States_Generated	States_Evaluated	States_Pruned	time
Minimax	1	116.3	116.3	-	0.0035
Minimax	2	573.9	427.5	-	0.0137
Minimax	3	1410.1	955	-	0.0249
Alpha-Beta	1	132.6	132.6	0	0.0058
Alpha-Beta	2	430.8	295.8	66.6	0.0131
Alpha-Beta	3	1050.1	657.4	156.5	0.0219
MCTS (Bonus)	-	7712.7	200	-	0.0712

3.4. Analysis of search depth

The experimental results show that increasing the search depth increases the computational cost of both Minimax and Alpha-Beta Pruning. For Minimax, the average number of generated states increases from 116.3 at depth 1 to 1,410.1 at depth 3, while the execution time increases from 0.0035 seconds to 0.0249 seconds. This indicates that deeper searches allow the algorithm to consider more possible future moves but require more computation.

Alpha-Beta Pruning reduces the number of states evaluated at greater search depths. At depth 2, it evaluates an average of 295.8 states compared with 427.5 for Minimax. At depth 3, it evaluates 657.4 states compared with 955 for Minimax, with an average of 156.5 pruning events. Alpha-Beta also has a lower average execution time at these depths: 0.0131 seconds at depth 2 and 0.0219 seconds at depth 3, compared with 0.0137 and 0.0249 seconds for Minimax, respectively.

However, at depth 1, Alpha-Beta is slower in the reported results, taking 0.0058 seconds compared with 0.0035 seconds for Minimax. Since no pruning occurs at this depth, the overhead of maintaining alpha and beta bounds may not provide a performance benefit.

Overall, the results suggest that Alpha-Beta Pruning becomes more beneficial as the search depth increases, reducing the number of evaluated states and execution time compared with Minimax at depths 2 and 3. Nevertheless, deeper search does not automatically guarantee better gameplay; its effectiveness also depends on the evaluation function and the game state. These results compare computational performance and do not, by themselves, establish which depth achieves the highest win rate.

4. Conclusion

In Single Player Mode, BFS, UCS, Greedy Best-First Search and A* collected all 5 apples in every game (Table 2.3.2). They differed in cost. Greedy Best-First Search expanded the fewest states (313.9 on average) and was the fastest (0.0053 seconds per game), followed by A* (801.0 states). BFS and UCS expanded more than 3,500 states, since they search without a heuristic. The bonus algorithm, IDS, also succeeded in every game but expanded far more states (129,576.75) and took the longest (1.2377 seconds), because it repeats its search at each depth limit. For this setting, the Manhattan-distance heuristic lowered the search cost without lowering the success rate.

In Multiplayer Mode, a deeper search increased the number of generated and evaluated states and the execution time of both Minimax and Alpha-Beta Pruning. Alpha-Beta Pruning evaluated fewer states than Minimax at depths 2 and 3, and it ran faster there, while at depth 1 it gave no benefit because nothing was pruned (Table 3.3.2). The bonus algorithm, MCTS, won all

10 games against Safe Random, but its random playouts make its choices close to random when every apple is far away. The results show a trade-off between search effort and decision quality.

5. Member Contributions

Table 5.1

Member Contributions

Member	Contribution	Signature
Ibnu Fikri (23526032)	Code: BFS, Alpha-Beta Pruning. Report: 2.2.2, 2.4.1, 2.4.5, 3.1, 3.2.2, 3.3.2, 3.4.	
Rabab Fawaz Saleh Alali (23526701)	Code: UCS, Minimax, IDS (bonus). Report: 2.2.1, 2.2.6, 2.4.1, 2.4.4,, 2.4.5, 3.1, 3.2.1, 3.3.1 , 3.3.2	
Zayed Ramadan Rahmat (23526015)	Code: Greedy Best-First Search, A*, MCTS (bonus). Report: 1, 2.1, 2.2.3, 2.2.4, 2.2.5, 2.3.1, 2.3.2, 2.4.2, 2.4.3, 3.2.3, 4, 5.	